



Carnegie Mellon University
Master of
Software Engineering

Quality attributes

LG Architecture Training Program

Paulo Merson

Quality attribute

*A quality attribute (QA) is a **measurable or testable property** of a system that is used to indicate how well the system satisfies the **needs of its stakeholders**.*
[Bass13]

*Quality attributes
are sometimes
referred to as "ilities"*

A few quality attributes

- Performance
 - Latency
 - Throughput
 - Timeliness
- Dependability
 - Availability
 - Reliability
 - Safety
- Interoperability
- Scalability
- Energy efficiency
- Security
 - Confidentiality
 - Integrity
 - Availability
- Modifiability
 - Modularity
 - Testability
 - Extensibility
- Usability
- Portability
- Deployability

Discussed later

Taxonomies of quality attributes

- There are published taxonomies, such as ISO/IEC 25010 and SEI [Barbacci95]
- They are informative, not prescriptive, and have limited practical use

In practice what is important is to specify the quality attribute requirement in a measurable/testable way

- There are two categories of attributes though
 - Properties of the system at **runtime**
 - Properties of the **development** of the system

How can you specify quality attribute requirements in a testable or measurable form?



Felix Bachmann

Quality attribute scenario

- A well-specified quality attribute scenario has six parts
 - **Stimulus**: condition affecting the system
 - **Source of stimulus**: entity that generated the stimulus
 - **Artifact**: artifact that was stimulated
 - **Environment/context**: condition under which the stimulus occurred
 - **Response**: activity undertaken as a result of the stimulus
 - **Response measure**: measure by which the system's response will be evaluated
- The scenario is a simple paragraph where you can identify these parts

*Sometimes
environment
is implicit*

The desired response should be measurable or testable

Performance scenario example

A smoke detector sends a smoke detected event to the building monitoring system at peak load. The system receives the event and triggers the sound alarm within 500 ms.

- **Stimulus**: smoke detected event
- **Source of stimulus**: smoke detector
- **Artifact**: building monitoring system
- **Environment/context**: at peak load
- **Response**: sound alarm is triggered
- **Response measure**: within 500 ms



Modifiability scenario example

*The PO requests the addition of a new payment gateway to the Checkout system.
The addition is created by the dev team and goes in production with an effort of 10 person-days.*

- **Stimulus**: request to add a new payment gateway
- **Source of stimulus**: product owner (PO)
- **Artifact**: Checkout system
- **Environment/context**: it's implicit that this is a change at development time
- **Response**: Checkout system is updated to offer the new payment gateway
- **Response measure**: 10 person-days effort



*GenAI can help
you to create QA
scenarios!*

Architecture tradeoffs

- When we improve a given quality in general, we can influence others...
 - In a positive way. Example: improved interoperability increases reliability
 - In a negative way. Example: improved confidentiality decreases performance
- A **design decision** may positively impact certain quality attributes while negatively affecting others
- That's an architecture **tradeoff**. Example:
 - Data replication improves availability and performance but has a negative effect on consistency



Tactic

- Established design technique that helps you achieve a quality attribute
- Examples:
 - *Encrypt data* is a security tactic
 - *Introduce concurrency* is a performance tactic
 - *Ping/echo* is an availability tactic
- In general, a tactic can be applied along with other tactics
- A tactic is simpler than a design pattern or architecture pattern



Pattern



- A design pattern is a proven solution to a common, recurrent problem
- Patterns are individually documented in a consistent format that contains:
 - Problem description and forces
 - Solution description with a reference structure of elements, relations and constraints
 - Impact and consequences of using the pattern
 - Application details and variations seen when implementing the pattern
 - Relations to other patterns
- Examples: Observer, Façade, Visitor, Model-View-Controller

Design pattern, architecture pattern and architecture style are related terms, but understanding the pattern/style itself is far more important than the classification

Mind the tradeoffs

- To use a given *tactic* or *pattern* is a *design decision* with positive and possibly negative effects on quality attributes, so...
- ... mind the tradeoffs!





Questions?

Paulo Merson
pmerson@cmu.edu